

Hybrid IoT and Computer Vision Framework for Real-Time Industrial Hazard Detection and Worker Safety Compliance

Sushant Tomar, Dr. Umesh Dutta, and Dr. Abhiruchi Passi
Manav Rachna International Institute of Research and Studies, Faridabad, India
Email: sushanttomar@ymail.com

Abstract—This paper presents a software-implemented hybrid framework for real-time industrial hazard detection and worker safety compliance. The framework integrates a YOLO-based computer vision module with a simulated Internet of Things (IoT) telemetry layer for gas, temperature, and vibration monitoring. Visual detections and sensor states are correlated using rule-based and confidence-weighted fusion to generate multi-level safety decisions. The prototype also supports event logging, dashboard-oriented runtime monitoring, and alert propagation through MQTT and email-ready interfaces. Unlike hardware-dependent prototypes, the proposed system can be tested using video input and programmatically generated sensor readings, enabling repeatable experimentation before field deployment. This revised paper removes implementation code listings and reports the system through architecture diagrams, runtime tables, and measured performance charts. In a 64-frame controlled video test, the system achieved an average throughput of 6.76 FPS, an average frame-processing latency of 160.38 ms, an average PPE-detection latency of 80.62 ms, and a decision distribution of 48 HIGH_RISK frames and 16 MEDIUM_RISK frames. The results demonstrate the feasibility of a software-first hybrid safety prototype and identify vision inference as the main runtime bottleneck.

Index Terms—Industrial Safety, Computer Vision, IoT Dashboard, YOLO, PPE Detection, Sensor Fusion, Simulated Sensors, MQTT, SQLite, Industry 4.0.

I. INTRODUCTION

Industrial workplaces such as manufacturing plants, construction sites, warehouses, and mining operations remain hazardous due to a combination of environmental, machine-related, and behavioral risks. Workers may be exposed to gas leakage, excessive temperature, abnormal vibration, moving machinery, restricted-zone intrusion, and personal protective equipment (PPE) non-compliance. Conventional supervision and CCTV-based monitoring are limited because they rely on continuous human observation and cannot automatically correlate invisible environmental hazards with visible unsafe actions.

Recent Industry 4.0 safety systems increasingly combine computer vision, IoT telemetry, and edge processing to improve responsiveness and situational awareness [1], [3], [6]. However, early-stage deployment is often slowed by hardware cost, sensor calibration, installation complexity, and the need for controlled validation. This work addresses that challenge through a software-first prototype that preserves the hybrid architecture while replacing physical sensors with simulated

telemetry. Such a design allows the fusion logic, alert flow, database logging, and dashboard behavior to be validated before real sensors are integrated.

The objective of this paper is to describe the system without embedding source-code listings in the manuscript. Instead, the implementation is summarized through modules, tables, and measured charts. This makes the paper more suitable for academic review because the results focus on system behavior, performance, limitations, and evaluation methodology rather than on raw program code.

II. RELATED WORK

A. Computer Vision for Safety Compliance

Computer vision has become a widely used approach for detecting helmets, safety vests, workers, vehicles, falls, and restricted-zone violations. Object-detection models such as YOLO, SSD, Faster R-CNN, and EfficientDet have been applied to PPE compliance and hazard recognition due to their ability to process video frames in real time [8], [7], [10]. In industrial settings, the performance of these models can degrade because of occlusion, low illumination, dust, camera angle, and variations in PPE color.

B. IoT and Simulated Sensing

IoT sensors support the detection of invisible hazards such as gas concentration, thermal anomalies, and vibration patterns. Sensor-based systems alone, however, cannot determine whether a worker is present, whether PPE is being worn, or whether a vehicle has entered a restricted zone. Simulation-driven sensor streams are useful during early-stage prototype validation because they allow repeatable testing of thresholds, alert rules, and data logging before physical hardware is connected [4], [12].

C. Sensor-Vision Fusion

Hybrid systems combine visual detections with sensor telemetry to improve reliability. Late fusion, where final sensor and vision outputs are combined by rules or weighted confidence scoring, is especially useful for safety systems because it is interpretable and easy to audit [6], [1]. A fused decision can distinguish between a harmless visual detection and a high-priority risk scenario, such as high gas levels in an occupied area or a worker without helmet protection near hazardous conditions.

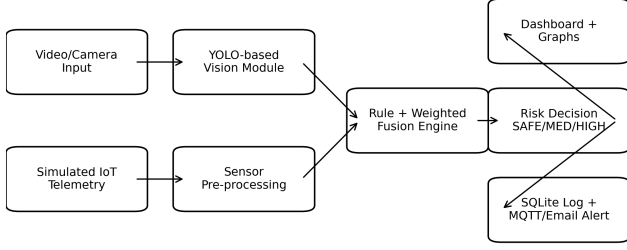


Fig. 1: Software-first hybrid safety architecture. Camera/video frames and simulated sensor telemetry are processed in parallel and combined by a fusion engine to generate safety decisions, logs, alerts, and dashboard outputs.

III. SYSTEM ARCHITECTURE

The proposed system consists of five major layers: video acquisition, computer vision inference, simulated IoT telemetry, fusion-based decision-making, and alert/logging output. Fig. 1 shows the revised architecture used in the prototype.

A. Vision Layer

The vision layer processes camera or video frames using a YOLO-based detector. The implementation is designed to load a custom PPE model when available and to use a fallback detector when the custom PPE model is absent. Detected objects include workers and PPE-related classes when supported by the model. In the current prototype, missing-helmet scenarios are treated as a major visual safety cue, and detection counts are logged per frame for runtime analysis.

B. Simulated IoT Layer

The sensing layer generates gas, temperature, and vibration values programmatically. These readings are not presented as physical measurements; rather, they represent simulated telemetry used to validate threshold checking, fusion rules, event logging, and graph generation. The approach is appropriate for prototype-level software evaluation and provides a repeatable way to test the behavior of the safety-decision pipeline.

C. Fusion and Alert Layer

The fusion engine combines sensor states and visual detections using rule-based conditions and weighted scoring. Risk decisions are expressed using operational classes such as MEDIUM_RISK and HIGH_RISK. Alerts can be propagated through MQTT and email-ready interfaces, while events are stored in a local database for audit and post-test analysis. This structure supports later extension to physical sensors and edge devices.

IV. REVISED IMPLEMENTATION DESCRIPTION

The earlier version of the paper included source-code listings for the vision module, sensor module, fusion engine, zone detection, SQLite logging, and main integration loop. These

TABLE I: Implementation Modules in the Revised Paper

Module	Role in the Prototype
Video acquisition	Reads frames from camera or video input and provides them to the vision pipeline.
YOLO-based detection	Detects worker/PPE-related objects and returns bounding boxes, labels, confidences, and detection counts.
Simulated telemetry	Generates gas, temperature, and vibration readings for repeatable software validation.
Fusion engine	Combines visual detections and sensor values into safety decisions using rules and weighted scoring.
Alert manager	Publishes risk events through MQTT/email-ready channels when configured.
Event logger	Saves decisions, sensor values, and vision meta-data for audit and analysis.
Metrics collector	Records FPS, latency, CPU, memory, detections, sensor values, and decisions per frame.
Graph generator	Produces publication-ready charts from the runtime metrics file.

TABLE II: Metric Scope and Reporting Status

Metric Type	Reported Here	Requirement
FPS and latency	Yes	Runtime frame log.
CPU and memory usage	Yes	Runtime system monitor.
Decision distribution	Yes	Predicted safety decisions.
Detection counts	Yes	Vision output per frame.
Precision, recall, F1-score	Not claimed	Labeled ground truth boxes/classes.
mAP@0.5	Not claimed	YOLO-format validation dataset.
mAP@0.5:0.95	Not claimed	Scenario-level true safety labels.
False alarm/missed hazard rate	Not claimed	

listings have been removed and replaced with a compact implementation summary. Table I describes the system modules at a research-reporting level.

V. EVALUATION METHODOLOGY

The system was evaluated using a frame-level runtime log generated during a controlled video-based test. The log contains 64 processed frames and includes timing measurements for individual pipeline stages, detection counts, sensor values, decisions, CPU utilization, and memory usage. The purpose of this experiment was to evaluate real-time feasibility and to identify bottlenecks in the software pipeline.

Accuracy metrics such as precision, recall, F1-score, and mean average precision (mAP) require ground-truth annotations for each frame or image. Since the available runtime log contains system predictions and runtime values but not manually labeled ground truth, this paper reports measured runtime performance and decision distribution. Table II separates measured metrics from ground-truth-dependent metrics.

VI. RESULTS AND DISCUSSION

A. Runtime Performance Summary

Table III summarizes the measured runtime values obtained from the 64-frame test. The results show that the system operated at an average of 6.76 FPS. The first frame included

TABLE III: Measured Runtime Performance Summary

Metric	Measured Value
Frames processed	64
Test duration	10.35 s
Average FPS	6.76
Median FPS	7.76
Minimum / maximum FPS	0.00 / 9.81
Average frame latency	160.38 ms
Median frame latency	121.26 ms
Maximum frame latency	330.39 ms
Average PPE detection latency	80.62 ms
Average CPU utilization	44.83%
Average memory usage	931.81 MB
Average PPE detections/frame	4.00

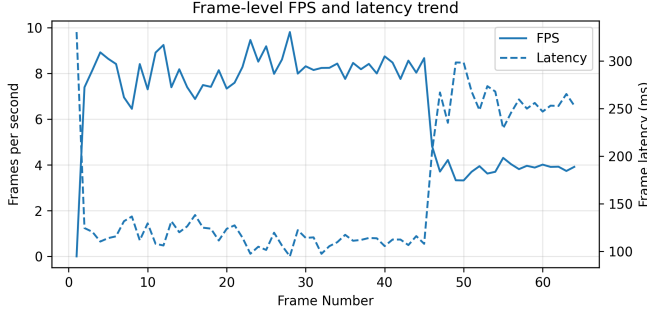


Fig. 2: Actual FPS and frame-processing latency over 64 frames. Higher latency corresponds to lower instantaneous FPS during later frames.

initialization overhead, producing a zero instantaneous FPS value; therefore, the median FPS of 7.76 is also reported to represent steady-state behavior more clearly.

Fig. 2 shows the frame-level relationship between FPS and processing latency. The first 45 frames generally maintain higher FPS and lower latency, while frames 46-64 show a marked latency increase and corresponding FPS reduction. This indicates that the pipeline may be sensitive to scene complexity, video source behavior, or display/processing overhead during later frames.

B. Pipeline Bottleneck Analysis

Table IV presents the average latency of each processing stage. PPE detection is the dominant contributor, accounting for the largest share of per-frame time. Frame acquisition, display, database logging, and annotation also contribute measurable overhead. In contrast, fusion, alert handling, fall detection, and zone detection are computationally lightweight.

C. Safety Decision Distribution

The system generated two risk classes during the test: HIGH_RISK and MEDIUM_RISK. As shown in Table V and Fig. 4, HIGH_RISK decisions occurred in 75% of frames, mainly due to worker-without-helmet detections and high combined multimodal anomaly scores. MEDIUM_RISK decisions accounted for the remaining 25% of frames and were associated with elevated but non-critical combined scores.

TABLE IV: Average Processing Time by Pipeline Stage

Stage	Average Time (ms)
PPE detection	80.62
Frame read	31.70
Display	18.23
Logging	14.99
Annotation	14.05
Sensor read	0.57
Alert handling	0.11
Fusion decision	0.02
Fall detection	0.001
Zone detection	0.001

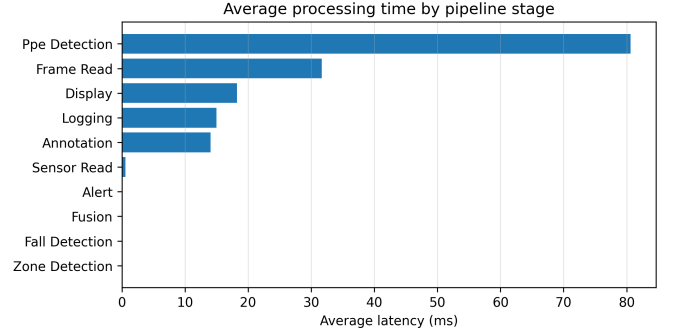


Fig. 3: Average processing time by pipeline stage. Vision inference is the primary bottleneck in the current software configuration.

D. Sensor and Resource Trends

The simulated telemetry values provided continuous environmental context for the fusion pipeline. The mean gas reading was 280.10 ppm, the mean temperature was 34.09 degrees C, and the mean vibration value was 130.64 in the configured simulation scale. Fig. 5 shows these time-series values over the processed frames. Although the values are simulated, the trend graph demonstrates how the dashboard and reporting layer can track environmental changes alongside vision-based detections.

CPU and memory trends are shown in Fig. 6. Average CPU utilization was 44.83%, and average memory usage was 931.81 MB. Memory increased during early frames and then stabilized near the upper range of the measured values, which is consistent with model loading, frame buffering, and runtime allocation behavior.

E. Interpretation

The results indicate that the current prototype is functionally suitable for controlled demonstration and research validation, but additional optimization is required before safety-critical deployment. The largest bottleneck is PPE detection latency, which suggests that future improvements should prioritize model optimization, frame skipping, image resizing, hardware acceleration, or edge-optimized inference. The fusion stage itself is negligible in runtime cost, confirming that decision-level fusion can be added without significantly affecting latency.

The decision distribution also shows that the tested scene contained persistent risk indicators, especially missing-helmet

TABLE V: Safety Decision Distribution

Decision	Frames	Share
HIGH_RISK	48	75.00%
MEDIUM_RISK	16	25.00%
SAFE	0	0.00%
CRITICAL_ALERT	0	0.00%
EMERGENCY	0	0.00%

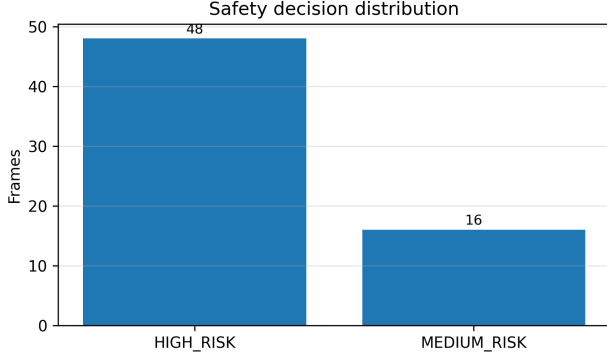


Fig. 4: Actual distribution of safety decisions for the 64-frame test.

detections. This is useful for demonstrating alert behavior, but future evaluation should include balanced scenarios containing SAFE, MEDIUM_RISK, HIGH_RISK, CRITICAL_ALERT, and EMERGENCY cases. Such balance is necessary for computing confusion matrices, false alarm rates, and missed hazard rates.

VII. ADDITIONAL EVALUATION RECOMMENDATIONS

To strengthen the research contribution, the next evaluation phase should use a labeled dataset. Table VI lists recommended validation additions. These additions would allow the prototype to move from runtime evaluation to full efficacy evaluation.

VIII. LIMITATIONS

The current study has four important limitations. First, the sensor values are simulated and therefore do not represent calibrated physical sensor behavior. Second, the available runtime log does not include ground-truth annotations, so accuracy metrics such as precision, recall, F1-score, and mAP are not claimed. Third, the test contains only 64 frames and should be extended to multiple videos and scenarios. Fourth, the computer vision model's PPE performance depends on the availability and quality of a domain-specific PPE model; generic fallback models may detect persons but may not reliably detect PPE violations.

IX. CONCLUSION

This paper presented a revised, code-free description of a hybrid IoT and computer vision framework for real-time industrial safety monitoring. The system integrates YOLO-based visual detection, simulated environmental telemetry, rule-based fusion, event logging, alerting, and frame-level

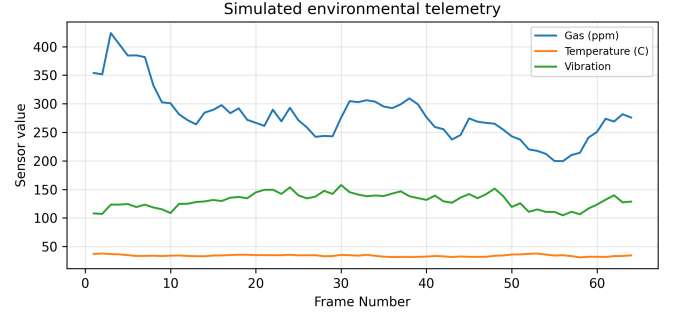


Fig. 5: Simulated gas, temperature, and vibration trends recorded during the runtime test.

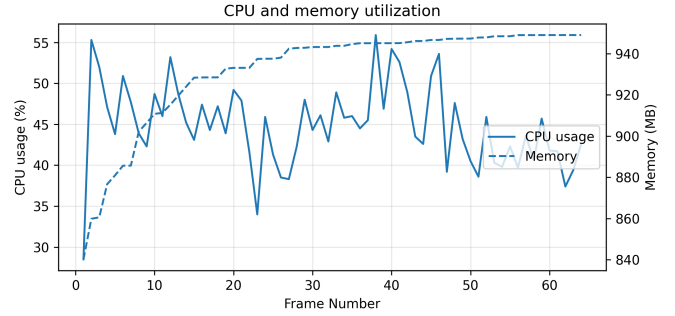


Fig. 6: CPU and memory utilization across the runtime test.

performance monitoring. Actual runtime results from a 64-frame test show an average FPS of 6.76, average frame latency of 160.38 ms, average PPE detection latency of 80.62 ms, average CPU utilization of 44.83%, and average memory usage of 931.81 MB. The results confirm that the software prototype can generate useful safety decisions and research-ready performance logs, while also identifying vision inference as the key optimization target. Future work will integrate physical sensors, add labeled datasets for full accuracy analysis, and benchmark the system on edge hardware for industrial deployment.

REFERENCES

- [1] A. Kalekar, R. Kadam, and S. Tamhane, "Hybrid AI-IoT System for Real-Time Worker Safety and Hazard Detection," *International Journal of Scientific and Advanced Technology*, 2025.
- [2] J. Dwyer et al., "Roboflow: A Platform for Computer Vision Dataset Management and Model Deployment," Roboflow Technical Report, 2023.
- [3] J. Park et al., "Artificial Intelligence and Smart Technologies in Safety Management," *MDPI Safety*, 2024.
- [4] U. Ejaz, W. Ramon, and M. Blessing, "IoT for Hazard Detection and Worker Safety Monitoring," ResearchGate, 2025.
- [5] C. O. Ozobu et al., "Advancing Occupational Safety with AI-Powered Monitoring Systems," *IIARD Journals*, 2025.
- [6] V. R. Uppaluri, "Real-Time AI-Driven Hazard Detection: Integrating Computer Vision and Sensor Networks," *IJSRCSEIT*, 2025.
- [7] R. Lan et al., "Computer Vision for Safety Management in the Steel Industry," *MDPI*, 2024.
- [8] P. Khandelwal, A. Khandelwal, and S. Agarwal, "Using Computer Vision to Enhance Safety of Workforce in Manufacturing Industries," *arXiv*, 2020.
- [9] J. Wu, S. Wu, and Y. Ma, "MonitorVLM: A Vision-Language Framework for Safety Violation Detection," *arXiv*, 2025.

TABLE VI: Recommended Additions for Full Efficacy Evaluation

Evaluation Item	Purpose
Annotated PPE dataset	Compute precision, recall, F1-score, and mAP for helmet/no-helmet/person classes.
Scenario labels per frame	Build a confusion matrix for SAFE, MEDIUM_RISK, HIGH_RISK, CRITICAL, and EMERGENCY decisions.
Safe baseline videos	Measure false alarm rate under normal working conditions.
Hazard videos	Measure missed hazard rate for PPE violation, gas anomaly, fall, and restricted-zone cases.
Hardware sensor trial	Validate that simulated sensor results generalize to MQ-series gas, DHT temperature, and vibration sensors.
Edge-device benchmark	Compare workstation, Jetson, Coral TPU, and Raspberry Pi/NPU performance.

[10] I. Yousif et al., “Safety 4.0: Harnessing Computer Vision for Advanced

Manufacturing Safety,” ScienceDirect, 2024.

- [11] “Artificial Intelligence in Manufacturing Industry Worker Safety,” MDPI, 2023.
- [12] M. J. Asif et al., “Leveraging Wireless Sensor Networks for Real-Time Monitoring,” arXiv, 2025.
- [13] “Real-Time Worker Safety Monitoring on Scaffolding: Integrating IoT with Computer Vision,” IRJAET, 2025.
- [14] S. Eiffert et al., “Toolbox Spotter: A Computer Vision System for Situational Awareness in Heavy Industries,” arXiv, 2021.
- [15] “Fall Prevention from Scaffolding Using Computer Vision and Smart Safety Hooks,” ASCE Library, 2025.
- [16] R. Konda, “Enhancing Workplace Safety with AI-Powered Video Surveillance,” Quest Journals, 2020.
- [17] “The Improvement of Industrial Safety Measures Using AI and IoT,” *Journal of Advanced Science and Technology*, 2025.
- [18] “Enhancing Industrial Safety and Risk Assessment Through AI-Powered Monitoring,” ESR Groups Journal, 2024.
- [19] “An Evaluation of Technologies for Real-Time Monitoring of Fall Risk in Construction,” MDPI, 2024.
- [20] J. Huber et al., “Evaluating User Safety Aspects of AI-Based Systems in Occupational Environments,” PMC, NCBI, 2025.